

# Structures de contrôle et les opérations avec des nombres à virgule flottante

BUT Informatique à Arles

Ricardo Uribe Lobello

# Les boucles de répétition

# L'instruction JMP

Les différentes variantes de l'instruction JMP permettent d'implanter des boucles de répétition. Par exemple, les instructions suivantes :

```
MOV    CL, 10
```

```
Boucle:
```

```
    <Instructions à répéter>
```

```
DEC    CL
```

```
JNZ    Boucle
```

Permettent d'exécuter un jeu d'instructions 10 fois. L'instruction JNZ teste le flag de non zéro, s'il n'est pas zéro, la boucle saute à nouveau à l'instruction Boucle.

# L'instruction Loop

Cette instruction assume que le registre ECX contient le compteur, dans ce cas, il exécute le code, ensuite, la valeur de ECX est décrémenté. Tant que ECX n'est pas arrivée à zéro, la boucle se poursuit. Ainsi, le code suivant :

```
MOV    ECX, 10
```

```
Boucle:
```

```
    <Instructions à répéter>
```

```
LOOP   Boucle
```

Est équivalent à celui de la diapositive précédente.

**Attention** : l'instruction loop ne marche qu'avec le registre ECX.

# RAPPEL : Les différents sauts conditionnels

Pour les valeurs numériques signées, les JUMPs suivants sont fournis :

| Instruction | Description                                               | Flags testés |
|-------------|-----------------------------------------------------------|--------------|
| JE/JZ       | Sauter si égale ou zéro                                   | ZF           |
| JNE/JNZ     | Sauter si non égale grand / Sauter si non zéro            | ZF           |
| JG/JNLE     | Sauter si plus grand / Sauter si non moins grand          | OF, SF et ZF |
| JGE/JNL     | Sauter si plus grand ou égale / Sauter si non moins       | OF et ZF     |
| JL/JNGE     | Sauter si moins grand / Sauter si non plus grand ou égale | OF et ZF     |
| JLE/JNG     | Sauter si moins grand ou égale / Sauter si non plus grand | OF, SF et ZF |

# RAPPEL : Les différents sauts conditionnels

Pour les valeurs numériques non signées, les JUMPs suivants sont fournis :

- Il s'agit de la même nomenclature mais avec Below au lieu de less et Above au lieu de Greater.

| Instruction | Description                                               | Flags testés |
|-------------|-----------------------------------------------------------|--------------|
| JE/JZ       | Sauter si égale ou zéro                                   | ZF           |
| JNE/JNZ     | Sauter si non égale grand / Sauter si non zéro            | ZF           |
| JA/JNBE     | Sauter si plus grand / Sauter si non moins grand          | CF et ZF     |
| JAЕ/JNB     | Sauter si plus grand ou égale / Sauter si non moins       | CF           |
| JB/JNAE     | Sauter si moins grand / Sauter si non plus grand ou égale | CF           |
| JBE/JNA     | Sauter si moins grand ou égale/ Sauter si non plus grand  | AF et CF     |

# RAPPEL : Les différents sauts conditionnels

Il existe des JUMPs permettant de tester directement les flags :

| Instruction | Description                          | Flags testés |
|-------------|--------------------------------------|--------------|
| JXCZ        | Sauter si CX est zéro                | Aucun        |
| JC          | Sauter s'il y a du carry             | CF           |
| JNC         | Sauter s'il n'y a pas du carry       | CF           |
| JO          | Sauter si débordement                | OF           |
| JNO         | Sauter s'il n'y a pas de débordement | OF           |
| JP/JPE      | Sauter si pair                       | PF           |
| JNP/JPO     | Sauter non pair                      | PF           |
| JS          | Sauter si la parité est négative     | SF           |
| JNS         | Sauter si la parité est positif      | SF           |

# La représentation des nombres



En assembleur, les nombres sont traités en binaire mais quand ils ont affichés ou récupérés de l'entrée, ils sont en ASCII.

C'est pour cela qu'il faut ajouter '0' aux nombres entre 0 et 9 avant de les afficher à l'écran et,

Si un nombre est saisi par l'utilisateur, il faut faire la soustraction de '0' avant de l'utiliser comme un entier.

Assembleur a deux manières de représenter les nombres :

- ASCII : comme déjà vu.
- BCD : Binary Coded Decimal.

**NOTE** : Nous parlerons plus sur cette représentation dans une de séances de TD.

# Les nombres à virgule flottante

A cause des raisons historiques, l'architecture x86 à 64 bits compte encore avec deux systèmes pour traiter des nombres à virgule flottante. Ces deux systèmes ont des capacités similaires mais aussi certaines différences. Ils sont :

- Le jeu d'instructions du coprocesseur x87 (le plus ancien) ;
- Le plus récent jeu d'instructions à traitement de vecteur (Vector Processing Instructions) ou XMM.

En assembleur, les nombres à virgule flottante utilisent :

- Un ensemble totalement différent des registres.
  - ST0 à ST7 pour l'ancien système coprocesseur x87 ;
  - XMM0 à XMM15 pour le nouveau XMM.
- Un jeu différent d'opérations.

# Les registres du coprocesseur X87

**Très important** : Avant de commencer à utiliser les registres et tout le système x87, il faut l'initialiser en utilisant l'instruction **EMMS**.

- Les registres utilisés sont ST0 à ST7 ;
- Ces registres sont stockés comme une pile afin de pouvoir rester (physiquement) le plus possible dans le coprocesseur. Par défaut, ST0 est dans la partie haute de la pile et les autres registres sont placés en ordre ascendant, ST0, ensuite ST1, après ST2 et ainsi de suite ;
- Les opérations ne prennent pas d'opérandes, l'opération sera appliquée sur les derniers deux éléments mis dans la pile ;
- Tous les nombres sont stockées avec une précision de 80 bits, même si les opérandes étaient à 64 bits de précision.

# La pile des registres du coprocesseur X87

Fonctionnement de cette pile :

- **fld** : Cette opération place des octets dans la pile. Ainsi, ce qui était en ST0 devient ce qui se trouve en ST1, ce qui était en ST1 est déplacé à ST2 et ainsi de suite ;
- **ST0** contiendra la valeur qui vient d'être déposée et ST0 sera marqué comme « in-use ».
- Par convention, les opérations **fld** ont été appelées «charger» (load en anglais).

`fld dword [addr]` ; Push **float** de la memoire

`fld qword [addr]` ; Push **double** de la memoire

`fld st1` ; Push st1 en st0

**;; Push un entier signé de taille dword de la mémoire.**

`fild dword [addr]`

**;; Push un entier signé de taille qword de la mémoire.**

`fild qword [addr]`

**;; Quelques opérations disponibles.**

`fld1` ; Push +1.0

`fldz` ; Push +0.0

`fldpi` ; Push  $\pi$

`fxch st3` ; Echange la valeur de st0 avec celle en st3.

# La pile des registres du coprocesseur X87

Fonctionnement de la pile :

- **fst** : Cette opération copie des éléments de la pile dans la mémoire. Ainsi, ce qui était en ST1 devient ce qui se trouve en ST0, ce qui était en ST2 est déplacé à ST1 et ainsi de suite ;
- Ce qui était en **ST0** est marqué comme libéré. **Attention** : faire un deuxième POP de **ST0** une fois libéré déclenchera une « stack underflow ».
- Par convention, les opérations **fst** sur cette pile ont été appelées «stocker» ou store en anglais.

**fst** dword [addr] ; Copie le **float** en st0 en [addr].

**fst** st1 ; Copie le **float** en st0 en st1.

**fstp** st1 ; Copie le **float** en st0 en st1 et fait POP.

;; Ecrit **float** ST0 comme un entier en addr.

**fist** dword [addr]

;; Ecrit le **float** ST0 comme un entier en addr et fait POP.

**fistp** dword [addr]

;; Ecrit un **double** trunqué comme entier en addr et fait POP

**fisttp** qword [addr]

# L'initialisation des nombres à virgule flottante

Ils peuvent être déclarés comme les nombres entiers en utilisant la définition de constantes, DB,DW, DD DQ, DT et DO.

Il faut les déclarer en utilisant un point pour séparer la partie entière de la décimale.

**Attention** : le point est obligatoire dans leur déclaration, même si le nombre a des décimales à 0. Par exemple :

**Reel DD 110.23**

**Un DB 1.0**

Il est aussi possible d'ajouter une **E(e)** afin d'exprimer une puissance à la fin tel que :

**Reel DD 1.1023e2**

|      |    |                         |                                            |
|------|----|-------------------------|--------------------------------------------|
| NR1  | db | -0.2                    | ; Réel avec un quart de précision.         |
| NR2  | dw | -0.5                    | ; IEEE 754r à moitié de précision.         |
| NR3  | dd | 1.2                     | ; IEEE 754 32 bits.                        |
| NR5  | dd | 0x1p+2                  | ; $1.0 \times 2^2 = 4.0$                   |
| NR6  | dq | 0x1p+32                 | ; $1.0 \times 2^{32} = 4\,294\,967\,296.0$ |
| NR7  | dq | 1.e10                   | ; 10 000 000 000.0                         |
| NR8  | dq | 1.e+10                  | ; C'est pareil que 1.e10                   |
| NR9  | dq | 1.e-10                  | ; Exprime 0.000 000 000 1                  |
| NR10 | dt | 3.141592653589793238462 | ; Valeur de PI                             |
| NR11 | do | 1.e+4000                | ; IEEE 754 precision étendue.              |

# Les opérations avec des réels

Les quatre opérations principales sont :

- **fadd** : addition ;
- **fsub/fsubr** : soustraction ;
- **fmul** : multiplication ;
- **fdiv/fdivr** : division.

Toutes ces opérations (op) peuvent être utilisées de plusieurs manières :

- op ST2 ; Signifiant  $ST0 = ST0 \text{ op } ST2$
- op ST0, ST2 : signifiant  $ST0 = ST0 \text{ op } ST2$
- op ST2, ST0 : signifiant  $ST2 = ST2 \text{ op } ST0$ .

Les opérations **fsubr** et **fdivr** permettent de faire les opérations avec les opérandes inversés. Ainsi, **fsubr ST0, ST1** veut dire  $ST0 = ST1 - ST0$  au lieu de  $ST0 = ST0 - ST1$ .

# Les opérations mathématiques avec des réels

Le coprocesseur x87 dispose de quelques opérations mathématiques.

**Pour quoi le savoir ?** : parce qu'une partie de ces opérations ne sont pas fournies par le nouveau système XMM.

Parmi elles il y a des fonctions :

- Trigonométriques ;
- La racine carrée ;
- La valeur absolue ;
- Les logarithmes.

**;; Fonctions trigonométriques.**

**fsin** ; st0 = sin(st0)

**fcos** ; st0 = cos(st0)

**fsincos** ; st0 = sin(st0), push cos(st0)

**fptan** ; st0 = tan(st0), push 1.0

**fpatan** ; st1 = atan(st1 / st0) and then pop

**;; La racine carrée.**

**fsqrt** ; st0 = sqrt(st0)

**fprem1** ; st0 = fmod(st0, st1) (fractional remainder)

**;; La valeur absolue.**

**fabs** ; st0 = |st0| (valeur absolue)

**;; Les logarithmes.**

**fyl2x** ; st1 = st1  $\times \log_2$ (st0)

**f2xm1** ; st0 =  $2^{\text{st0}} - 1$



# Le nouveau système XMM

## Extension SSE (Streaming SIMD Extension)

Ce système moderne partage des ressources avec le coprocesseur x87 et avec l'unité vectorielle « Single Instruction, Multiple Data (SIMD) ».

- Il n'y a pas besoin d'être initialisé comme le système x87 ;
- Il utilise les registres dédiés XMM0 à XMM15 :
  - Tous ses registres sont à 128 bits mais il est possible de les utiliser avec 32 (précision simple) et 64 bits (précision double).
- XMM0 à XMM7 sont utilisés pour passer d'arguments flottantes aux fonctions ;
- XMM8 à XMM15 sont des registres temporaires. Ils peuvent être sur-écrits par un appel à fonction. Il faut les enregistrer avant l'appel si nous voulons les conserver ;

# Le opérations dans XMM

Ce système possède aussi son propre jeu d'instructions. Les opérations de déplacement sont :

- **MOVSS** destination, source : Déplace des flottantes à précision simple (32 bits) ;
- **MOVSD** destination, source : Déplace des flottantes à précision double (64 bits).

**Attention** : A nouveau, Il n'est pas possible de déplacer deux opérandes en mémoire. Il faut qu'un soit un registre.

**Important** : Contrairement à l'instruction MOV traditionnelle, il n'est pas possible d'affecter directement des littéraux Il faut que la source soit en mémoire.

# Changement de précision en XMM

Il existe des fonctions pour faire le changement de précision entre nombres à virgule flottante :

| Instruction        | Description                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------|
| cvtss2sd dest, src | Conversion de 32 bit à 64 bit flottante. Src peut être en mémoire, un registre flottant ou un registre général. |
| cvtss2sd dest, src | Conversion de 64 bit entier à 32 bit flottante.                                                                 |
| cvtss2si dest, src | Conversion de 32 bit flottante à 32 bit entier signé. Dest doit être un registre général ou flottant.           |
| cvtss2si dest, src | Conversion de 64 bit flottante à 32 bit entier.                                                                 |
| cvtss2ss dest, src | Conversion de 32 bit entier signé à 32 bits flottante. Dest doit être un registre flottant.                     |
| cvtss2sd dest, src | Conversion de 32 bit entier signé à 64 bits flottante.                                                          |
|                    |                                                                                                                 |

# Les opérations en XMM

Pour les opérations arithmétiques de base en XMM :

- Les deux opérandes doivent être des registres XMM ;
- Le résultat de l'opération sera stockée dans l'opérateur destination ;
- La taille des opérandes doit être la même.

|                        |                                 |
|------------------------|---------------------------------|
| <b>addss</b> dest, src | ; dest += src ( <b>float</b> )  |
| <b>addsd</b> dest, src | ; dest += src ( <b>double</b> ) |
| <b>subss</b> dest, src | ; dest -= src ( <b>float</b> )  |
| <b>subsd</b> dest, src | ; dest -= src ( <b>double</b> ) |
| <b>mulss</b> dest, src | ; dest *= src ( <b>float</b> )  |
| <b>mulsd</b> dest, src | ; dest *= src ( <b>double</b> ) |
| <b>divss</b> dest, src | ; dest /= src ( <b>float</b> )  |
| <b>divsd</b> dest, src | ; dest /= src ( <b>double</b> ) |

# Les opérations en XMM

Il existe des versions des opérations arithmétiques avec trois opérandes. Pour ces opérations :

- Elles sont préfixées avec un **v** ;
- Destination et src1 doivent être des registres XMM mais src2 peut être en mémoire ;
- Le résultat de l'opération sera stockée dans l'opérateur destination ;
- La taille des opérandes doit être la même.

**vaddss** dest, src1, src2 ; dest = src1 + src2

**vaddsd** dest, src1, src2 ; dest = src1 + src2

**vsubss** dest, src1, src2 ; dest = src1 - src2

**vsubsd** dest, src1, src2 ; dest = src1 - src2

**vmulss** dest, src1, src2 ; dest = src1 \* src2

**vmulsd** dest, src1, src2 ; dest = src1 \* src2

**vdivss** dest, src1, src2 ; dest = src1 / src2

**vdivsd** dest, src1, src2 ; dest = src1 / src2

# Fonctions mathématiques en XMM

XMM implémente un ensemble restreint d'opérations mathématiques :

**Attention** : les opérations trigonométriques ne sont pas implémentées.

**; La racine carrée.**

**sqrtps** dest, src

**sqrtsd** dest, src      dest = sqrt(src)

**rsqrtps** dest, src      dest = 1/sqrt(src)

**; Ratio inverse.**

**rcpps** dest, src      dest = 1/src

**;; Maximum et minimum entre deux opérandes.**

**maxss** dest, src

**maxsd** dest, src      ; dest =maximum entre dest et src

**minss** dest, src

**minsd** dest, src      ; dest =minimum entre dest et src

**;; Arrondi d'un flottant en utilisant un mode choisi**

**roundss** dest, src, mode ;Round src into dest using mode

**;; Les modes d'arrondissement disponibles.**

**Mode** = 0 ; Vers le plus proche.

**Mode** = 1 ; Vers le bas.

**Mode** = 2 ; Vers le haut.

**Mode** = 3 ; Arrondi vers zéro.

# Les opérations de comparaison

Il existe des instructions différentes pour comparer des nombres flottantes. Néanmoins :

- Elles écrivent leur résultats dans les mêmes flags que les opérations à nombres entiers non signés ;
- Ainsi, les opérations de saut :  
**JE/ENE** : Egale ou non égale,  
**JA/JAE** : (plus grand) Above ou égale  
et **JB/JBE** : (moins grand) Below ou égale vont marcher.

;; Fait la comparaison entre dest et src. dest doit  
;; être un registre à nombre flottant.

**ucomiss** dest, src ; Pour 32 bits.

**ucomisd** dest, src ; Pour 64 bits.

;; **Observation** : Pour utiliser printf, il faut remplir  
;; AL avec le nombre d'arguments utilisés avec  
;; des registres XMM.

**mov** rdi, format

**mov** al, 1

;; Converti à double pour printf.

**cvtss2sd** xmm0, xmm0 **call** printf

# Exemple d'utilisation des nombres à virgule flottante : Calcul de PI

```
section .data  
zero dd 0.0  
un dd 1.0  
deux dd 2.0  
quatre dd 4.0  
negone dd -1.0  
limite dd 0.000001  
format db "La valeur approximée de PI est %f", 10, 0
```

```
section .text  
extern printf  
                global main  
main:  
    push RBP  
    mov RBP, rsp  
    ;; Fonction pour calculer PI  
    call calcul_PI  
    ; Le retour de la fonction sera en XMM0
```



# Exemple d'utilisation des nombres à virgule flottante : Calcul de PI

**;; Afficher le résultat.**

**mov** RDI, format

**mov** al, 1

**;;** Fait la conversion de float à double pour printf.

**cvtss2sd** XMM0, XMM0

**call** printf

**mov** rax, 0

**pop** RBP

**ret**

**calcul\_PI:**

**push** RBP

**mov** RBP, RSP

**movss** XMM7, dword [un] ; 1.0

**movss** XMM0, dword [zero] ; p = 0

**movss** XMM1, XMM7 ; s = 1

**movss** XMM2, XMM7 ; d = 1 ; XMM3 = t

**Boucle:**

**movss** XMM3, XMM7 ; t = 1

**divss** XMM3, XMM2 ; t /= d

**vmulss** XMM4, XMM1, XMM3 ; XMM4 = s \* t

**addss** XMM0, XMM4 ; p += s \* t

**mulss** XMM1, **dword** [negone] ; s \*= -1

**addss** XMM2, **dword** [deux] ; d += 2

**;;** Tant que t > limite

**ucomiss** XMM3, **dword** [limite]

**ja** Boucle

**;;** Il faut déposer le résultat dans XMM0

**mulss** XMM0, **dword** [quatre]

**pop** RBP

**ret**

# Les chaînes de caractères

En assembleur, les chaînes de caractères peuvent avoir autant de caractères que nécessaire.

Par contre, il est nécessaire de connaître sa longueur, pour cela, il est possible de :

- Stocker directement sa longueur ;
- Ajouter un caractère de fin de chaîne.

Pour connaître la longueur, nous pouvons utiliser le pointeur de location **\$** qui pointe, à ce moment là, à l'octet suivant à celui de la fin de notre chaîne de caractères.

; Cette chaîne est stocké avec le caractère de passage à la ligne suivante à la fin (code ascii 10).

msg **db** 'Bonjour à tous', 10 ; Le message.

len **equ** \$ - msg ; La longueur du message.

; Cette chaîne est stocké avec un caractère de fin de chaîne null qui a le code ASCII **0**.

message **DB** 'C'est un autre message', 0

# Les tableaux (arrays)

Il est possible de déclarer des tableaux de données en utilisant les directives DX.

Ces nombres sont stockés dans des cases consécutives de deux bytes (un mot).

La position du premier nombre est Nombres, le deuxième sera [Nombres + 2] et ainsi de suite.

**Très important** : il faut faire les déplacements par octet et pas par rapport à la taille des éléments du tableau.

; Déclaration d'un tableau avec 6 nombres

Nombres **DW** 5, 6, 10, 12, 15, 20

; Déclaration d'un tableau avec 5 zéros.

Ages **DW** 0, 0, 0, 0, 0

; Pour faire de manière plus compacte.

Ages **TIMES 5 DW** 0

# Les tableaux (arrays)

Un exemple de somme des nombres dans un tableau.

## section .data

```
T DD 2, 4, 6, 8, 19
```

## section .bss

```
    somme resd 1
```

...

```
MOV eax, 5 ; On va faire la somme de 5 nombres
```

```
MOV ebx, 0 ; EBX contiendra la somme.
```

```
;; ECX va pointer vers l'élément courant
```

```
MOV ecx, T
```

## Boucle:

```
ADD ebx, [ecx] ; Ajoute le prochain nombre.
```

```
;;Déplace le pointeur vers l'élément suivant.
```

```
ADD ecx, 4
```

```
DEC eax      ; Décrementation du compteur.
```

```
JNZ Boucle  ; Si le compteur n'est pas  
              ; zéro, on répété la boucle.
```

## Fin :

```
;;Résultat stocké en somme.
```

```
MOV [somme], ebx
```

# Le fonctionnement de la pile (the stack)

**La pile** est un des concepts le plus fondamental dans la programmation en général mais elle a une importance particulière en assembleur. Ses caractéristiques sont :

- C'est une structure dans une place protégée de la mémoire dans laquelle on peut déposer des données ;
- La pile est une structure de type LIFO (Dernier en entrer, premier en sortir). Cela veut dire que le dernier élément à être stocké sera le premier à être enlevé.

Les deux instructions en assembleur pour manipuler la pile sont :

- **PUSH** <opérande> : Dépose le contenu de l'opérande en haut de la pile ;
- **POP** <Adresse/registre> : Enlève le donnée en haut de la pile et le stocke dans l'adresse ou dans le registre fourni comme opérande.

# Implémentation de la pile

- **segment stack** : contient l'implémentation de la pile.
- **La taille des éléments** dans la pile dépend de la quantité de bits de l'architecture dans laquelle est exécuté le programme. Dans 64 bits, chaque élément de la pile occupe au plus 64 bits.
- **RSP, ESP, SP** : Ces registres pointent vers le dernier élément à avoir été ajouté à la pile. C'est à dire, celui du tout haut.
- **RBP, EBP, BP** : Ces registres pointent vers la base de la pile.
- **SS** : Ce registre pointe vers le début du segment de la pile.

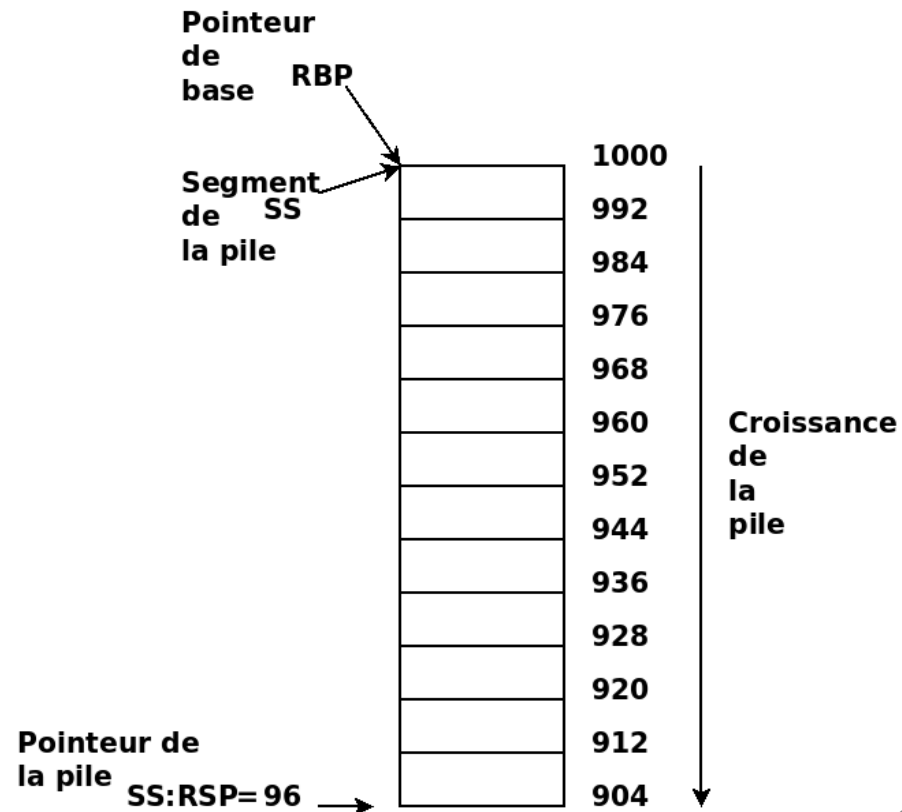
Par convention, les adresses de la pile avancent en descendant, cela veut dire que le dernier élément ajouté aura une adresse plus basse que celle de l'élément lui précédent.

**Attention** : RSP, ESP et SP donnent un décalage au sein du segment de la pile, ils ne pointent pas vers une adresse absolue.

# Fonctionnement de la pile

Par convention, les adresses de la pile avancent en descendant, cela veut dire que le dernier élément ajouté aura une adresse plus basse que celle de l'élément lui précédent.

- **RBP** : pointe vers la base de la pile et le début du segment de la pile (SS).
- **RSP** : Pointe vers la prochaine adresse de la pile.





# Les registres à conserver dans la pile (1/3)

Les registres suivants devraient être enregistrés dans la pile avant de les utiliser, des faire des appels aux procédures ou aux fonctions parce qu'ils peuvent être sur-écrits dans une de ces opérations.

| Registres (64, 32, 16, 8 bits) | Explication                                                                                                       |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| RCX, ECX, CX, CH et CL         | C'est un registre qui est parfois sur-écrit. Il est parfois utilisé comme un compteur pour quelques instructions. |
| RDX, EDX, DX, DH et DL         | Quelques instructions l'utilisent comme compteur.                                                                 |
| RBX, EBX, BX, BH et BL         | C'est réservé Il faut toujours le sauver avant de l'utiliser.                                                     |
| RBP, EBP, BP et BPL            | C'est réservé pour stocker la partie base de la pile.                                                             |
| RSI, ESI, SI et SIL            | Utilisé pour passer le deuxième paramètre d'une fonction C.                                                       |
| RDI, EDI, DI et DIL            | Utilisé pour passer le première paramètre d'une fonction C.                                                       |

# Les registres à conserver dans la pile (2/3)

Les registres suivants ont été tous ajoutés en 64 bits mais ils sont parfois utilisés par NASM sans informer l'utilisateur. Il faut les enregistrer dans la pile avant de les utiliser.

| registres (64, 32, 16, 8 bits) | Explication                                                                    |
|--------------------------------|--------------------------------------------------------------------------------|
| R8, R8d, R8w et R8b            | Ajouté en 64 bits avec une nomenclature différente.                            |
| R9, R9d, R9w et R9b            | Ajouté en 64 bits avec une nomenclature différente.                            |
| R10, R10d, R10w et R10b        | Ajouté en 64 bits avec une nomenclature différente.                            |
| R11, R11d, R11w et R11b        | Ajouté en 64 bits avec une nomenclature différente.                            |
| R12, R12d, R12w et R12b        | C'est réservé. Il est possible de l'utiliser mais il faut l'enregistrer avant. |
| R13, R13d, R13w et R13b        | C'est réservé. Il est possible de l'utiliser mais il faut l'enregistrer avant. |
| R14, R14d, R14w et R14b        | C'est réservé. Il est possible de l'utiliser mais il faut l'enregistrer avant. |
| R15, R15d, R15w et R15b        | C'est réservé. Il est possible de l'utiliser mais il faut l'enregistrer avant. |

# Les registres à conserver dans la pile (3/3)

Par convention, il est nécessaire que le code appelant la fonction (caller) et le code appelé (callee) soient chacun responsable d'une partie du code. En général:

- Le caller doit être responsable de conserver les registres suivants :
  - RAX, RCX, RDX, RSI, RDI, R8–R11.
- Le callee doit conserver les registres suivants:
  - RBX, RBP, R12–R15.

**Mais attention:** cela n'est qu'une convention et parfois d'autres registres doivent être protégés.

**Règle à suivre:** Quand vous commencez à avoir des erreurs de « **segmentation fault** », il est probablement parce qu'il faut mettre dans la pile un de ces registres.

# Les conventions d'appel (pour les systèmes Linux) (1/4)

Les machines Linux 64 bits utilisent l'interface « **AMD 64 System V Application Binary** » pour les appels aux fonctions.

Cette convention est très différente par rapport aux précédentes, dans lesquelles il fallait stocker les arguments dans la pile en ordre inverse.

## Les arguments de la fonction

**Arguments entiers** : Les premiers six arguments doivent se trouver dans les registres : RDI, RSI, RDX, RCX, R8, R9 dans cet ordre exact.

Si plus d'arguments sont nécessaires, ils doivent être stockés dans la pile dans l'ordre inverse par rapport à leur apparition dans la signature de la fonction.

**Arguments en point virgule** : Les premiers arguments réels sont dans les registres xmm0, xmm1, xmm2, xmm3, xmm4, xmm5, xmm6 et xmm7.

# Les conventions d'appel (pour windows) (2/4)

Néanmoins, les machines Windows 64 bits utilisent l'interface « **Application Binary Interface x64** » pour les appels aux fonctions. Cette convention est aussi différente par rapport aux précédentes mais elle ressemble, à celle du **AMD 64 System V Application Binary** avec certaines différences importantes.

## Les arguments de la fonction

**Uniquement quatre paramètres sont fournis dans des registres. Pour les :**

**Arguments entiers :** les registres utilisés sont RCX, RDX, R8 et R9 dans cet ordre exact.

**Arguments en point virgule :** les registres utilisés sont xmm0, xmm1, xmm2 et xmm3.

Si plus d'arguments sont nécessaires, ils doivent être stockés dans la pile dans l'ordre inverse par rapport à leur apparition dans la signature de la fonction.

# Les conventions d'appel (3/4)

## L'instruction de retour

Avant d'appeler la fonction ou procédure, assembler stocke l'adresse de la prochaine instruction dans la pile.

## La valeur de retour

La valeur retournée par la fonction sera systématiquement stockée dans le registre RAX.

## Les registres préservés

Les registres RSP (Pointeur à la pile), RBP (Pointeur de base), RBX et R12 à R15 peuvent être modifiés pendant l'appel à la fonction.

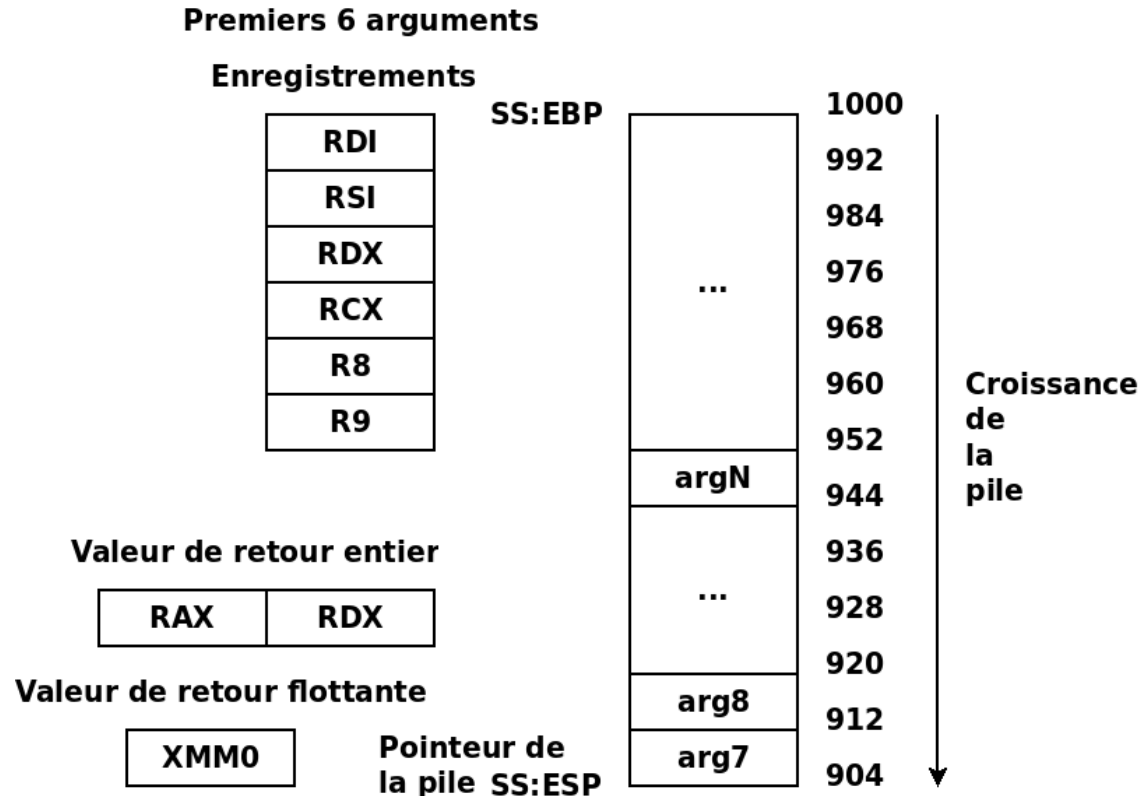
**Très important** : En principe, c'est la responsabilité de la fonction qui est appelée (en assembleur) d'enregistrer tous ses registres (dans la pile) avant de les utiliser et de les restaurer avant de retourner le contrôle avec **ret**.

Si la fonction est C, le programme Assembleur appelant peut stocker ces registres.

Si cela n'est pas fait un **segmentation fault, stack segment mis-alignment** peut être déclenché.

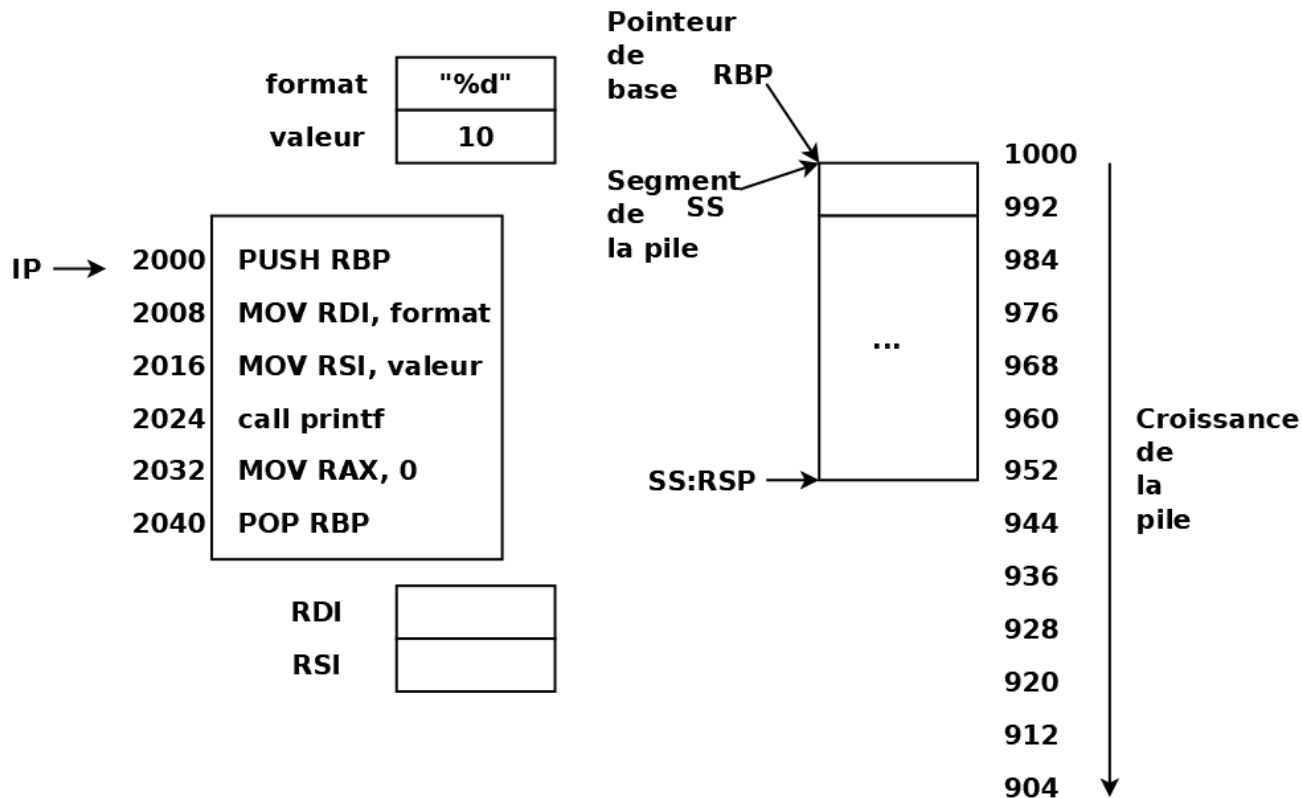
# Les conventions d'appel (pour la convention AMD) (4/4)

## La structuration des arguments et des valeurs de retour



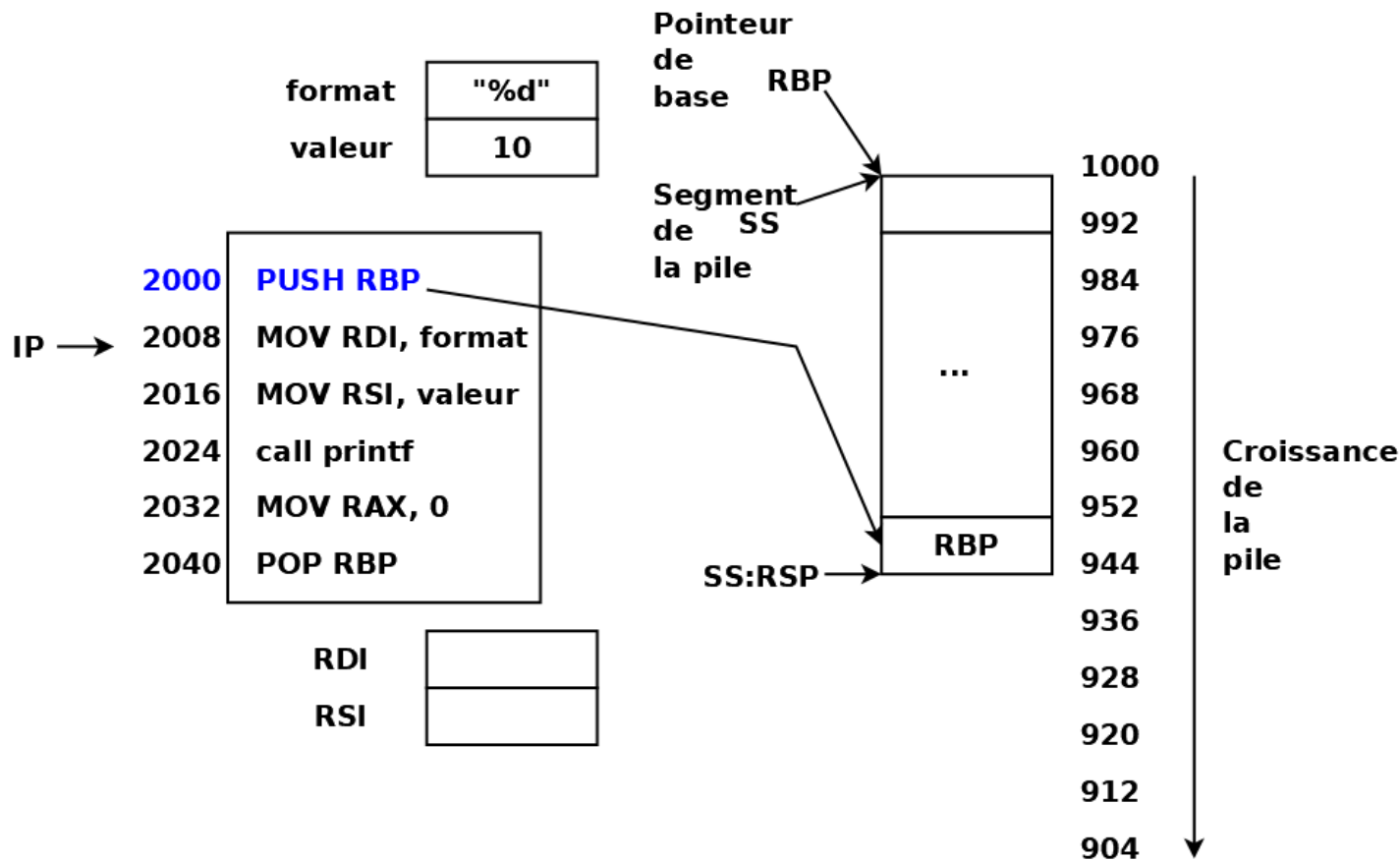
# Exemple d'un appel à une fonction

## La structuration des arguments et des valeurs de retour

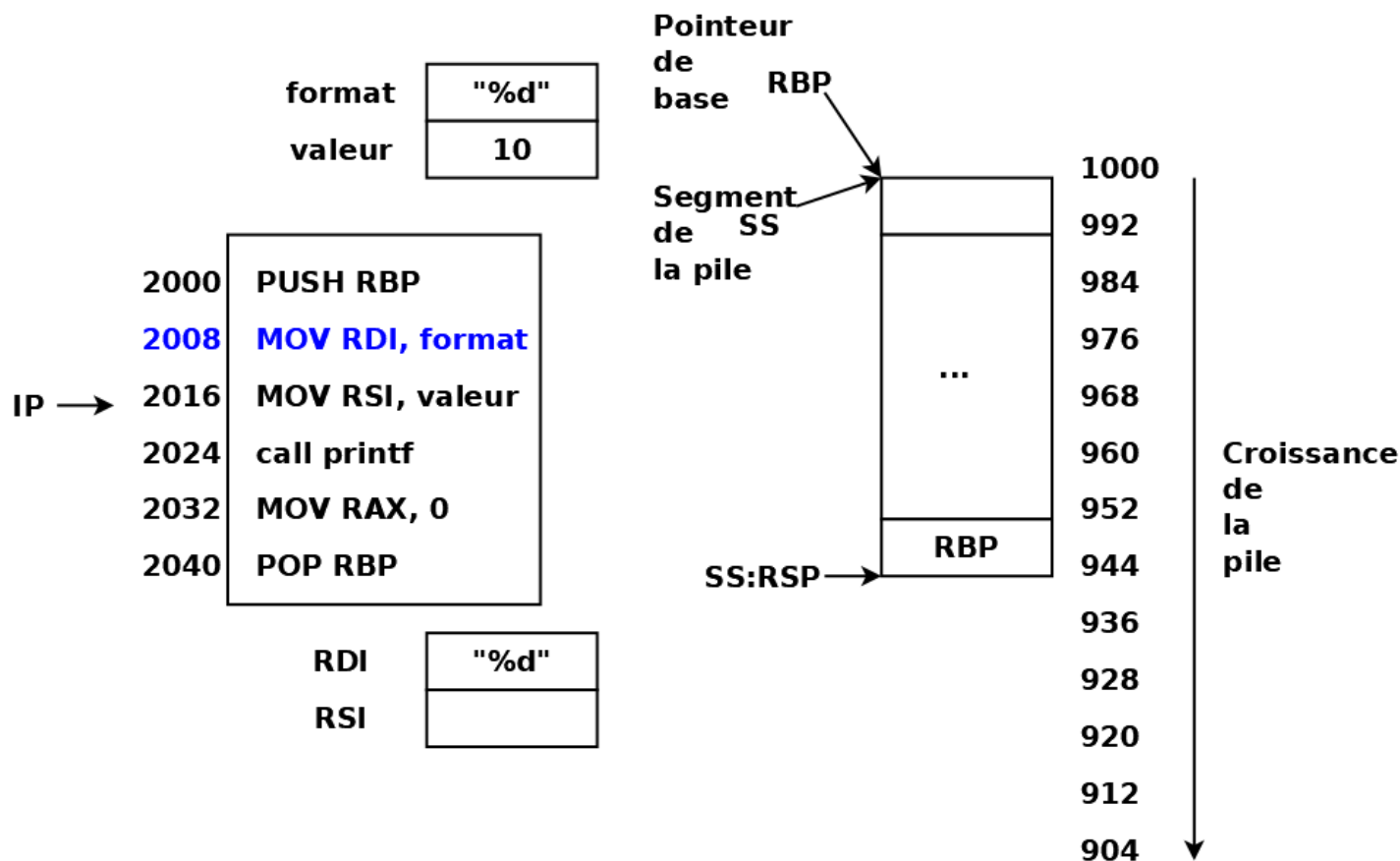




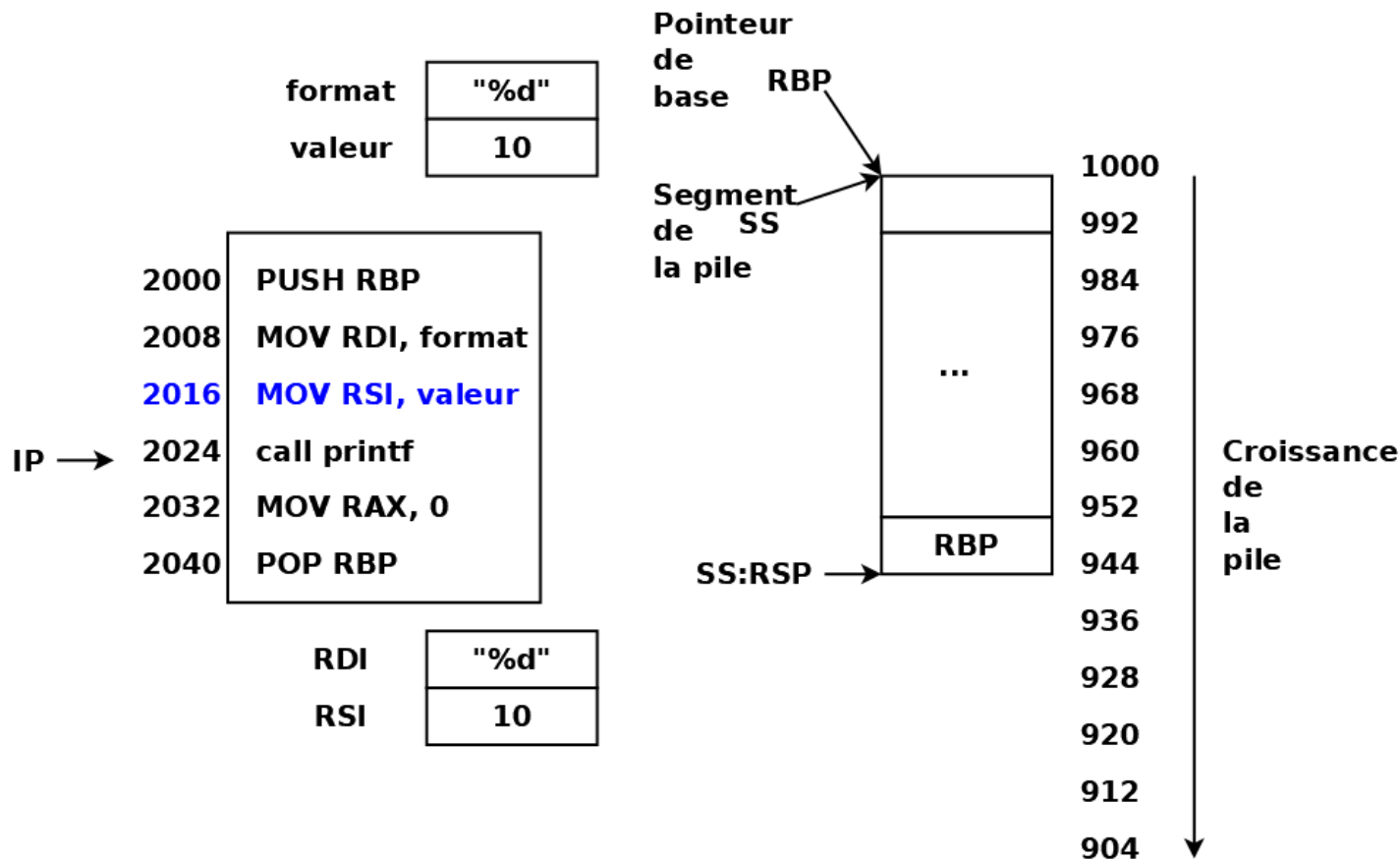
# Exemple d'un appel à une fonction



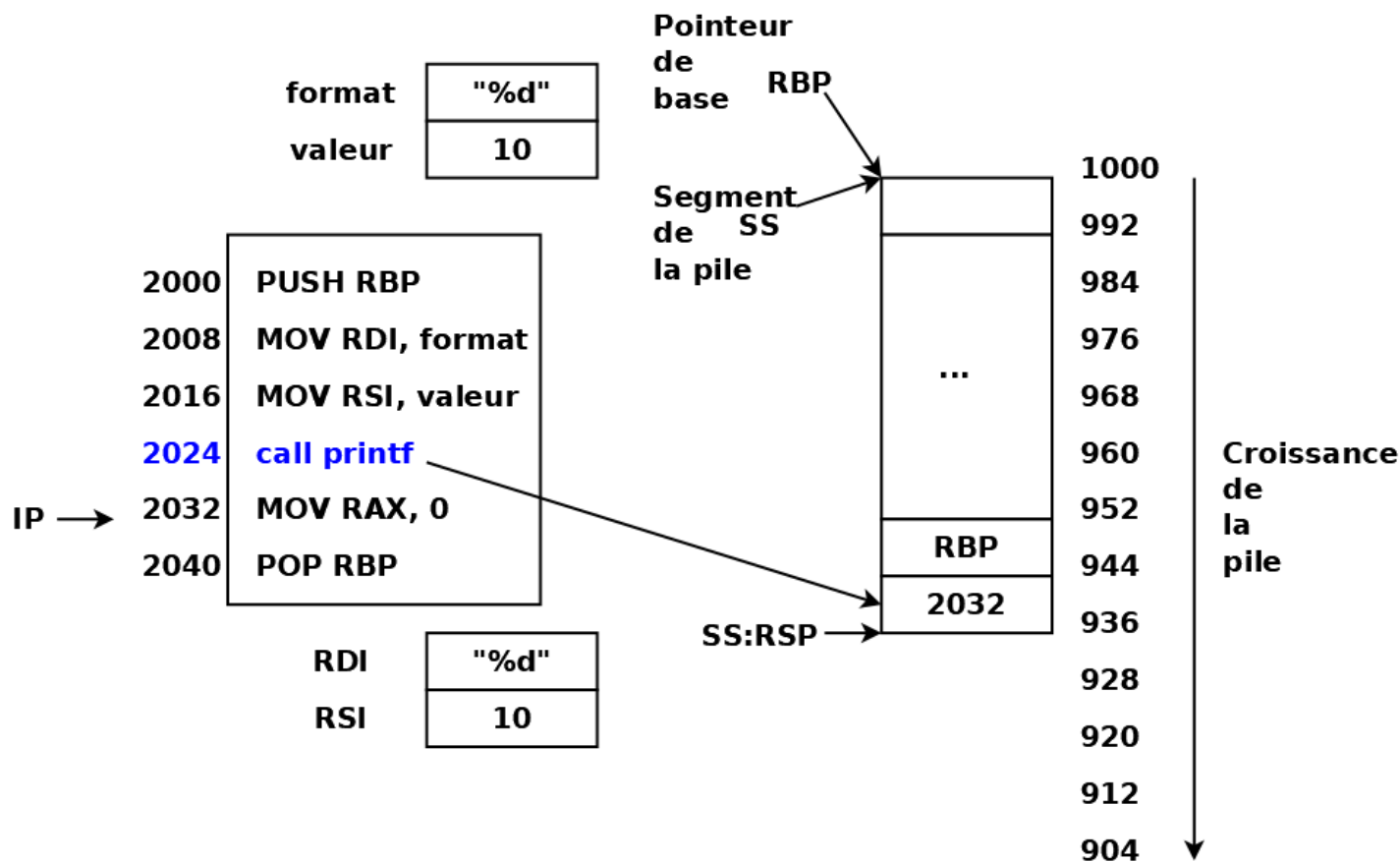
# Exemple d'un appel à une fonction



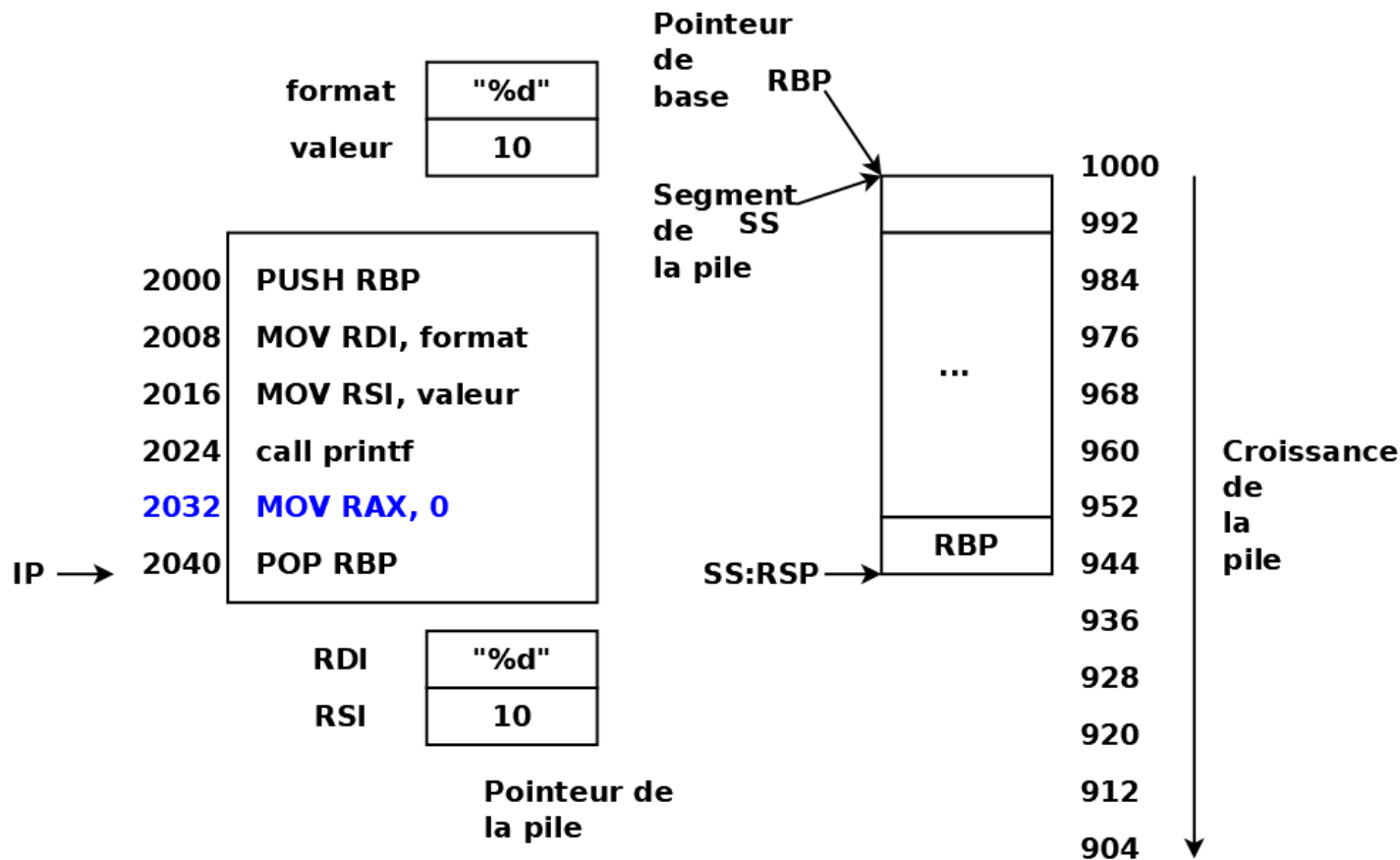
# Exemple d'un appel à une fonction



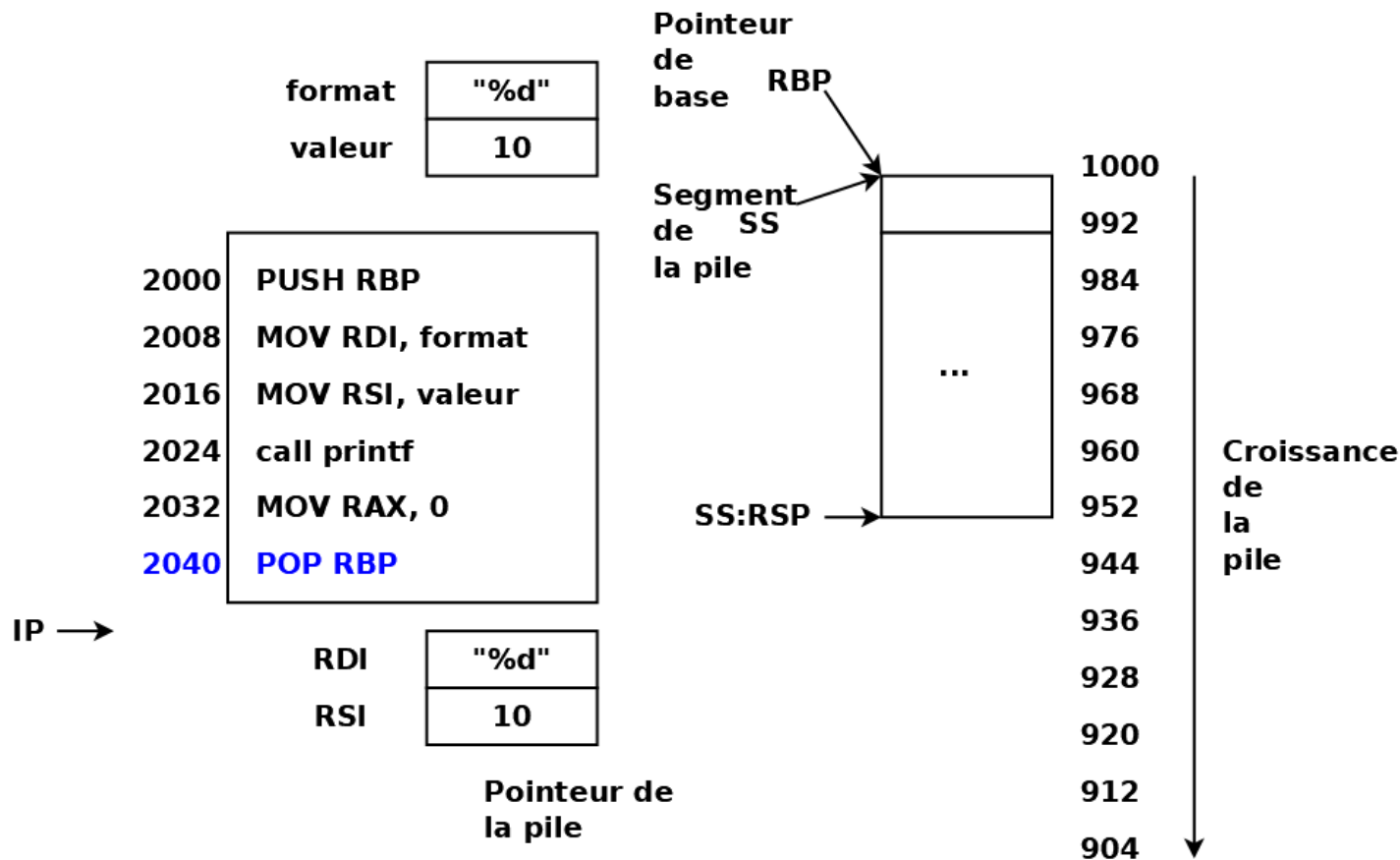
# Exemple d'un appel à une fonction



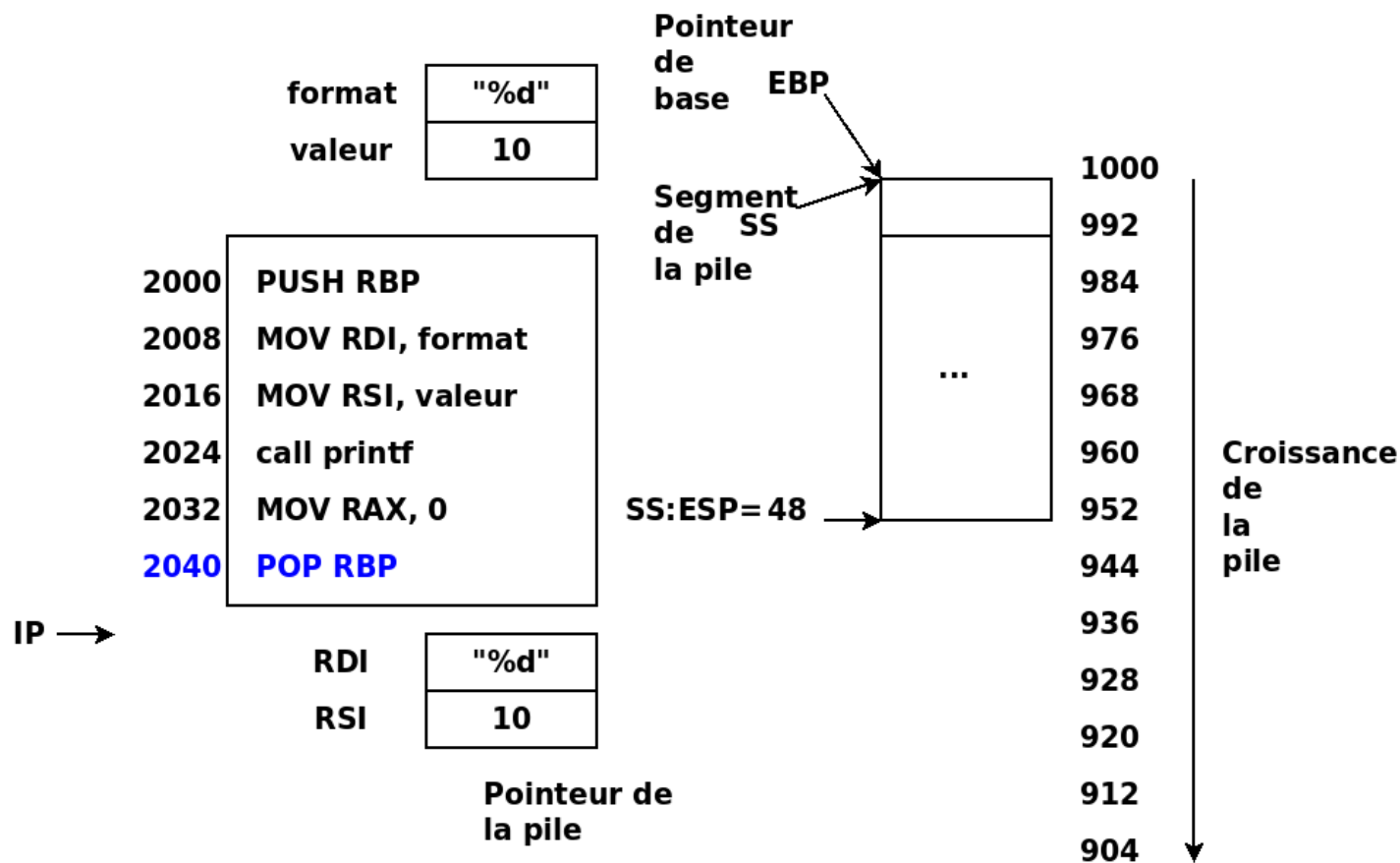
# Exemple d'un appel à une fonction



# Exemple d'un appel à une fonction



# Exemple d'un appel à une fonction



# Règles pour l'appelé (callee) et pour l'appelant (caller)

| Pour le programme appelant                                                                                            | Pour la fonction/procédure appelée                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Stocker les registres qui risquent d'être modifiés.                                                                   | Si des variables locales sont utilisées, stockez de la mémoire utilisant SUB RSP, <nombre_bytes> parce que la pile croît en réduisant les adresses. |
| Les 6 premiers paramètres sont passés dans des registres et les suivants en ordre inversé dans la pile                | Stockez les registres RBX, RBP, et R12 à R15 avant de les utiliser.                                                                                 |
| Appel la fonction avec call. L'adresse de l'instruction suivante sera stockée dans la pile comme l'adresse de retour. | Placez la valeur de retour en RAX avant de faire ret.                                                                                               |
| Une fois la fonction retournée, il faut enlever les paramètres qui ont été mis dans la pile.                          | Récupérez les valeurs des registres RBX, RBP, et R12 à R15 dans l'ordre inverse avant de retourner.                                                 |
| Vérifiez la valeur de retour déposé en RAX.                                                                           | Libérez les variables locales en faisant ADD RSP, <nombre_bytes>                                                                                    |
| Faites POP de tous les registres stockés dans la pile avant l'appel à la fonction.                                    | Retournez le contrôle en utilisant <b>ret</b> .                                                                                                     |



# Les procédures et fonctions

**Procédure en assembleur** est un bloc de code avec une fonctionnalité spécifique. Il est identifié avec un nom unique au sein du programme.

L'appel à une procédure transfère le contrôle du programme à la procédure.

Une fois la procédure finie, elle rend le contrôle au programme après la ligne dans laquelle elle a été appelée.

**Objectifs** : Ils servent à mieux organiser le code et à éviter le besoin de répéter le code des fonctionnalités qui sont réalisées plusieurs fois. Par exemple, les entrée/sorties.

; Structure de base d'une procédure

Nom\_de\_la\_procedure:

<Bloc de code de la procédure>

...

Ret ; Instruction de retour et fin de la procédure.

; Appel à la procédure.

Call Nom\_de\_la\_procedure

# Les procédures et fonctions

Dans cet exemple, la procédure `somme` permet de faire la somme de nombres dans les registres ECX et EDX :

```
section .text
global _start
_start:
    mov     ecx, '4'
    sub     ecx, '0'
    mov     edx, '5'
    sub     edx, '0'
    call    somme           ;Appel à la procédure
somme
    mov     [res], eax
    mov     ecx, msg
    mov     edx, len
    mov     ebx, 1          ;stdout
    mov     eax, 4          ;sys_write
    int     0x80            ;Appel au kernel
```

```
    mov     ecx, res
    mov     edx, 1
    mov     ebx, 1          ;stdout
    mov     eax, 4          ;sys_write
    int     0x80            ;Appel au kernel

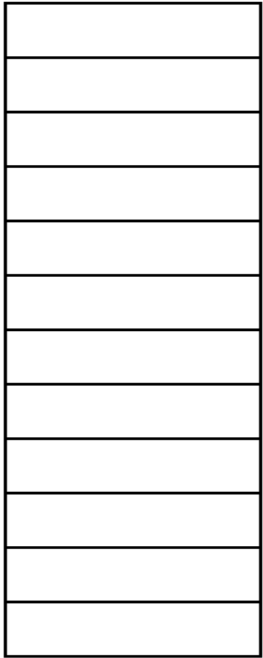
    mov     eax, 1          ;sys_exit
    int     0x80            ;Appel au kernel

somme:
    mov     eax, ecx
    add     eax, edx
    add     eax, '0'
    ret

section .data
msg db "La somme est:", 10
len equ $- msg
segment .bss
res resb 1
```

# Exemple d'appel à une fonction (1/4)

**Mémoire Haute**



**Mémoire Basse**

```
#include <iostream>

int somme (int a, int b);

int main (void) {
    int c = 1, d = 4;
    int x = somme(c,d);
    std::cout << x << std::endl;
    return 0;
}

int somme (int a, int b) {
    int c = 0;
    c = a + b;
    return c;
}
```

# Exemple d'appel à une fonction (2/4)

## Mémoire Haute

|             |
|-------------|
| a = 1       |
| b = 4       |
| Adresse ret |
| Ancien RBP  |
| RSI         |
| RDI         |
| RBX         |
| c = 5       |
|             |
|             |
|             |
|             |

## Mémoire Basse

```
PUSH RBP
MOV RSP, RBP
```

```
;; Sauver des enregistrements.
PUSH RSI
PUSH RDI
PUSH RBX
```

```
;; Créer une variable locale c.
SUB ESP, 8
```

```
;; Met b en EBX
MOV EBX, [RBP + 24]
```

```
;; Fait la somme a + b.
ADD EBX, [RBP + 16]
```

```
;; Fait c = a + b
MOV [RBP - 32], EBX
```

```
;; Retourner le résultat en RAX.
MOV RAX, [RBP - 32]
```

← RBP

← RSP

```
#include <iostream>
```

```
int somme (int a, int b);
```

```
int main (void) {
```

```
    int c = 1, d = 4;
```

```
    int x = somme(c,d);
```

```
    std::cout << x << std::endl;
```

```
    return 0;
```

```
}
```

```
int somme (int a, int b) {
```

```
    int c = 0;
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

# Exemple d'appel à une fonction (3/4)

## Mémoire Haute



RBP  
RSP

```
PUSH RBP
MOV RSP, RBP

;; Sauver des enregistrements.
PUSH RSI
PUSH RDI
PUSH RBX

;; Créer une variable locale c.
SUB ESP, 8

;; Met b en EBX
MOV EBX, [RBP + 24]

;; Fait la somme a + b.
ADD EBX, [RBP + 16]

;; Fait c = a + b
MOV [RBP - 32], EBX

;; Retourner le résultat en RAX.
MOV RAX, [RBP - 32]

;; Nettoyage de la pile.
ADD ESP, 8
POP RBX
POP RDI
POP RSI
```

## Mémoire Basse

```
#include <iostream>
```

```
int somme (int a, int b);
```

```
int main (void) {
```

```
    int c = 1, d = 4;
```

```
    int x = somme(c,d);
```

```
    std::cout << x << std::endl;
```

```
    return 0;
```

```
}
```

```
int somme (int a, int b) {
```

```
    int c = 0;
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

# Exemple d'appel à une fonction (4/4)

## Mémoire Haute



## Mémoire Basse

```
PUSH RBP
MOV RBP, RSP

;; Sauver des enregistrements.
PUSH RSI
PUSH RDI
PUSH RBX

;; Créer une variable locale c.
SUB ESP, 8

;; Met b en EBX
MOV EBX, [RBP + 24]

;; Fait la somme a + b.
ADD EBX, [RBP + 16]

;; Fait c = a + b
MOV [RBP - 32], EBX

;; Retourner le résultat en RAX.
MOV RAX, [RBP - 32]

;; Nettoyage de la pile.
ADD ESP, 8
POP RBX
POP RDI
POP RSI
MOV RSP, RBP
POP RBP
RET
```

```
#include <iostream>
```

```
int somme (int a, int b);
```

```
int main (void) {
```

```
    int c = 1, d = 4;
```

```
    int x = somme(c,d);
```

```
    std::cout << x << std::endl;
```

```
    return 0;
```

```
}
```

```
int somme (int a, int b) {
```

```
    int c = 0;
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

# A se rappeler

- Les boucles de répétition sont implémentés avec des étiquettes et des instructions de saut conditionnel ;
- L'existence de deux systèmes de calcul avec des nombres à virgule flottante, préprocesseur x87 et système XMM ;
  - Ces deux systèmes coexistent parce que le coprocesseur x87 fourni des opérations trigonométriques non fournies dans le système XMM ;
- La convention d'appel C à 64 bits permet de gérer l'interaction du code assembleur avec du code dans d'autres langages mais pas que cela, elle est fondamentale pour pouvoir écrire du code assembleur ;
- Le mécanisme de la pile permet de définir des contextes locaux à l'intérieur de chaque procédure ou fonction en assembleur (ou n'importe quel autre langage), elle est fondamentale pour comprendre l'interaction du code venant des différentes librairies ou programmes.